

Service Identification Table

University ERP · Microservice Architecture · Advanced Software Engineering

1. Purpose

This document identifies all eight microservices that compose the University ERP system. Each service is independently deployable, owns its own database, and communicates with other services via synchronous REST (HTTP/JSON) or asynchronous Kafka events. Service boundaries are drawn around distinct business capabilities of the academic domain.

2. Service Identification Table

Service	Port	Business Capability	Owned Data	DB	Communication
Auth Service	3007	Authentication & authorisation — issues and validates JWTs	Users, Tokens	auth_db	REST (consumed by all services)
Student Service	3001	Manage student records and enrolment	Students, Enrolments	students_db	REST (provider); Kafka producer: student.created
Course Service	3002	Manage courses and modules	Courses, Modules	courses_db	REST (provider)
Exam Service	3003	Create exams and register student exam entries	Exams, Entries	exams_db	REST (calls Student Service); Kafka producer: exam.registered
Result Service	3004	Record and publish exam results and grades	Results, Grades	results_db	REST (provider); Kafka producer: result.published
Transcript Service	3005	Generate and store student PDF transcripts	PDF store, Metadata	transcripts_db	REST (calls Result Service); Kafka consumer
Notification Service	3006	Send email/SMS notifications on key events	Notification Log	notif_db	Kafka consumer: student.created, exam.registered, result.published
Reporting	3008	Aggregate	Snapshots,	reports_db	Kafka consumer:

Service		metrics and generate snapshots for admin portal	Metrics		result.published
---------	--	---	---------	--	-------------------------

3. Service Boundary Rules

- Each service owns exactly one MongoDB database — no service reads another service's database directly.
- Cross-service data access is performed exclusively via REST API calls or Kafka events.
- JWT tokens issued by Auth Service are validated independently by each service's own middleware.
- Shared identifiers (e.g. studentId) are passed between services as references, not joined data.
- NGINX acts as a reverse proxy routing external requests by URL path prefix — it is not a smart API gateway.

4. Inter-Service Communication Overview

4.1 Synchronous (REST / axios)

Caller	Called Service	Endpoint	Purpose
Exam Service	Student Service	GET /api/students/:id	Verify student exists before exam enrolment
Transcript Service	Result Service	GET /api/results/:studentId	Fetch grades to generate PDF transcript

4.2 Asynchronous (Kafka)

Event Topic	Producer	Consumers
student.created	Student Service	Notification Service
exam.registered	Exam Service	Notification Service
result.published	Result Service	Notification Service, Reporting Service, Transcript Service

5. Technology Stack (All Services)

Layer	Technology	Rationale
Runtime	Node.js 20 (LTS)	Single language across all services; large ecosystem
Framework	Express.js	Minimal, flexible HTTP server; well-understood
ODM / DB Driver	Mongoose + MongoDB	Schema-flexible; one cluster, isolated databases
Authentication	jsonwebtoken + bcrypt	Stateless JWT; no session state needed
Sync HTTP client	axios	Promise-based; used for cross-service REST calls
Async events	kafkajs	Kafka with kafkajs for Node.js
PDF generation	pdfkit	Used only by Transcript Service
Reverse Proxy	NGINX	Free, lightweight path-based routing; no vendor lock-in
Containerisation	Docker + docker-compose	Reproducible environments; one-command startup

Note: The prototype (Prototype) implements Student Service and Exam Service only. The remaining six services are designed here but will be noted as "designed, not implemented" in the submission.